

Navegación: Pila vs. Pestañas

Diferencia conceptual y cómo está aplicado (o no) en GymBro App

1. Navegación por Pila (Stack Navigation)

Es una estructura **LIFO** (Last In, First Out) — literalmente una pila, como una pila de platos o el call stack de un programa.

- Cada vez que navegás “hacia adelante” (`navigation.navigate('X')` o `push('X')`), la nueva pantalla se **apila arriba** de la anterior.
- Cada vez que volvéis atrás (`goBack()`, gesto de swipe, botón físico atrás en Android), se **desapila** la pantalla de arriba, revelando la que estaba debajo.
- Existe una **relación jerárquica/secuencial** entre pantallas: $A \rightarrow B \rightarrow C$ implica que B es “hija” de A en el recorrido, y C es “hija” de B. Hay una noción de “de dónde vine” que persiste.
- Es el patrón natural para flujos de **profundización** (drill-down): lista de productos $\rightarrow$ detalle de producto $\rightarrow$ editar producto. Tiene sentido volver atrás paso a paso por ese camino.
- Visualmente suele animarse como pantallas que se deslizan desde el costado (una entra, la anterior se desliza pero sigue “debajo”).

2. Navegación por Pestañas (Tab Navigation)

Es una estructura **paralela/lateral**, no jerárquica.

- Varias secciones de primer nivel **coexisten** — no hay una “encima de otra”, son pares entre sí (Inicio, Rutinas, Cuenta... ninguna es “hija” de otra).
- Cambiar de pestaña **no apila nada**: es simplemente “ahora estoy mostrando esta sección en vez de aquella otra”. No hay push ni pop.
- Normalmente **cada pestaña mantiene su propio historial interno independiente** (en React Navigation, cada tab puede tener su propio Stack anidado adentro). Si entrás profundo en la pestaña A, saltás a la B, y volvéis a la A, la app te muestra exactamente dónde habías quedado en A — no te resetea al inicio de esa pestaña.
- No existe un “atrás” entre pestañas: pasar de Inicio a Rutinas no genera una entrada de historial que puedas deshacer con el botón atrás — atrás afecta al historial interno de la pestaña activa, no al orden en que tocaste las pestañas.
- Todas las secciones son accesibles **directamente en cualquier momento** vía la barra fija (a diferencia del stack, donde solo podés ir “hacia adelante” a donde el flujo te lleva, o “hacia atrás” por donde ya viniste).

Comparación rápida

	Stack	Tabs
Estructura	LIFO (pila)	Secciones paralelas
Relación entre pantallas	Jerárquica (padre/hijo)	Pares, sin jerarquía

Cambiar de sección	Apila una pantalla nueva	Cambia qué sección se muestra
Historial	Único, compartido	Uno independiente por pestaña
Botón atrás	Desapila paso a paso	Afecta solo a la pestaña activa
Caso de uso típico	Lista → detalle → editar	Inicio / Rutinas / Cuenta, etc.

3. Cómo está aplicado (o no) en GymBro App

Esta app usa navegación por pestañas **visualmente**, pero por debajo es **100% navegación por pila**.

Evidencia en el código

```
const Stack = createNativeStackNavigator(); // app/index.tsx
```

Es la **única** clase de navegador que se usa en todo el proyecto. Nunca se importa `createBottomTabNavigator` de `@react-navigation/bottom-tabs` (que sería el navegador real de pestañas de React Navigation).

`TrainerBottomNav` y `UserBottomNav` **no son un Tab Navigator** — son un View con `TouchableOpacities`, cada uno llamando:

```
onPress={() => navigation.navigate('mi_rutina_user')}
```

Ese `navigation` viene del **mismo** `Stack.Navigator` que declara todas las rutas del grupo “usuario común”. Entonces, aunque visualmente parece una barra de tabs (iconito + texto, resaltado en amarillo, fija abajo), **funcionalmente cada botón está empujando/saltando dentro de una pila**, no cambiando de una sección paralela independiente a otra.

Consecuencias prácticas de esa decisión

- **Comportamiento de `navigate()` en un stack:** si la pantalla destino ya está en la pila, React Navigation salta hacia atrás hasta esa pantalla (descartando lo de arriba) en vez de apilar una copia nueva; si no está, la apila.
- **El botón atrás / gesto de swipe sigue activo:** si el usuario toca Inicio → Rutinas → Reloj → Mi Cuenta y después usa el botón físico atrás, retrocede pantalla por pantalla por ese camino, en vez de comportarse como una barra de pestañas real.
- **No hay headers:** `headerShown: false` oculta la flecha “atrás” nativa que delataría que es un stack, por eso visualmente se siente como tabs aunque el mecanismo sea pila.
- **El resaltado de la pestaña activa es manual:** cada pantalla pasa `activeScreen="home"` a mano a la barra, porque el componente no tiene forma de saber “en qué pestaña estoy” solo (no hay pestañas reales).

4. ¿Conceptualmente estaría mejor con Tabs?

Sí — la barra de abajo le dice al usuario “estas son secciones paralelas de tu cuenta, podés saltar directo a cualquiera”, y eso es semánticamente navegación por pestañas, no por pila. Usar un Tab Navigator real alinearía el mecanismo con lo que la interfaz ya promete:

- Cada sección mantendría su propio historial independiente (volver a “Rutinas” te dejaría donde habías quedado, no en su raíz).
- El botón atrás se comportaría como se espera de una app con tabs (no acumularía los saltos entre secciones como pasos para deshacer).

- El estado “pestaña activa” lo manejaría React Navigation solo, sin depender de que cada pantalla pase `activeScreen` a mano.
- Resolvería la duplicación entre `TrainerBottomNav` y `UserBottomNav`: un `Tab.Navigator` con `tabBarIcon/tabBarLabel` por `screen` reemplaza ambos componentes.

Por qué no es un error grave dejarlo así en este proyecto

- Es un proyecto académico con datos mockeados, no una app en producción con navegación profunda sostenida — el costo de no preservar historial por pestaña es bajo.
- Migrar a `Tab Navigator` implica reestructurar el árbol: un `Tab.Navigator` con un `Stack` anidado dentro de cada `tab` (para las pantallas que sí necesitan drill-down, como “Rutinas” → “Ver día X”). No es un cambio de una línea.
- El proyecto ya resuelve el caso de uso principal de forma funcional, aunque no sea la implementación “libro de texto”.

Respuesta recomendada para el oral

“Conceptualmente el patrón correcto para esto es un `Bottom Tab Navigator`, con un `Stack` anidado dentro de cada `tab` para las pantallas de profundización como el detalle de un día de rutina. Lo implementamos con `Stack` + una barra manual porque para el alcance del proyecto no necesitábamos la preservación de historial por sección, pero reconocemos que un `Tab Navigator` sería la solución más prolija y sería el primer refactor si la app creciera.”